MODULE 07 · DOMAIN 6 - PROMPT AND CONTEXT ENGINEERING

Output Handling

Structured outputs, schema design, validation, defensive parsing, and a healthy distrust of confident answers.

DOMAIN 6 of 8 EXAM WEIGHT 11.0% MODULE 7 of 36

admin@sthithapragna.com

When code reads the answer, hope is not a parser.

Asking for JSON or guaranteeing it?

PROMPT-ONLY JSON

Usually right

- Format requested in words
- Mostly complies
- Sometimes extra prose or a broken bracket
- You parse, and hope

STRUCTURED OUTPUTS

Shape guaranteed

- Schema sent in the request
- Constrained decoding
- "Always valid: no more JSON.parse() errors"
- Or strict tools for tool inputs

THE TIEBREAKER

If a program consumes the output, use structured outputs. Asking nicely is not a format guarantee.

A structured output request

```
1  "output_config": {"format": {
2    "type": "json_schema",
3    "schema": {
4      "type": "object",
5      "properties": {
6        "urgency": {"type": "string", "enum": ["emergency", "urgent", "routine"]},
7        "trade": {"type": "string", "enum": ["plumbing", "heating", "other"]},
8        "summary": {"type": "string"}},
9      "required": ["urgency", "trade", "summary"],
10     "additionalProperties": false}}}
```

6-7 Fixed categories as enums: no new spellings, no new labels.

9-10 Every field required; nothing extra allowed.

THE TAKEAWAY

In Python, messages.parse() takes a Pydantic model, sends the schema and returns a typed object.

Designing the schema

- **Enums for fixed categories,** so labels cannot drift.
- **Allow "not stated":** make a field nullable when the source may be silent, or Claude will fill it anyway.
- **Descriptive names, flat shapes:** deposit_amount_gbp beats amt; avoid needless nesting.
- **No personal data in the schema itself.** Schemas are cached; PHI must never appear in schema definitions.
- **Version the schema** with the code that consumes it.

> ON THE EXAM
>
> A required field with no way to say "unknown" is an invitation to invent a value.

What structured outputs do not promise

- **A refusal** takes precedence over the schema, so the output may not match.
- **Stopped at max_tokens:** the JSON may be incomplete.
- **An enum value** may differ from your schema only in capitalisation.
- **Right shape, wrong value:** a valid field can still hold an invented number.
- **Also:** the first use of a schema adds latency while it compiles; not combinable with citations or prefill.

> BE PRECISE
>
> Structured outputs remove parse errors. They do not remove the need to check.

Structured outputs guarantee the **shape**.
Validation checks the **meaning**.

admin@sthithapragna.com

Validate before anything downstream

01

Stop reason

Was it end_turn? Refusal or max_tokens fails here.

02

Parse

Into a typed object; errors become values, not crashes.

03

Schema

Required fields, types and enums, normalised for case.

04

Business rules

Dates, limits, totals, cross-field checks.

05

Route

Pass on, or send to retry, repair or fallback.

THE TAKEAWAY

Nothing reaches the database, the invoice or the tenant until all four checks pass.

Defensive parsing in code

```python
def read_extraction(response) -> Extraction | Failure:
    if response.stop_reason != "end_turn":
        return Failure(f"stopped early: {response.stop_reason}")
    try:
        data = Extraction.model_validate_json(first_text(response))
    except ValidationError as err:
        return Failure(f"schema: {err}")
    if data.deposit_gbp and data.deposit_gbp > 5 * data.weekly_rent_gbp:
        return Failure("deposit above five weeks' rent")
    return data
```

2-3 Check why generation stopped before reading anything.

6-7 A schema error becomes a value, never a crash.

8-9 A business rule no schema can express.

THE TAKEAWAY

Every path returns something the caller can act on. Nothing is silently dropped.

Defensive parsing: the habits

- **Missing fields:** a deliberate default, or the failure path. Never a guess.
- **Unexpected fields:** ignore or reject on purpose, and log them.
- **Type mismatches:** coerce only when it is safe, such as "12" to 12; otherwise fail.
- **Case and whitespace:** normalise before comparing with enums.
- **Never** crash the downstream system, and never silently drop output.

> ON THE EXAM
>
> Defensive parsing means every odd input has a planned outcome.

When validation fails

RESPONSE	WHAT IT MEANS
Retry	Send the validation error back to Claude and ask for a corrected output. Cap the attempts.
Repair	Fix trivial issues in code, such as case or whitespace, then re-validate.
Fall back	Route to a person, a simpler path, or a safe default.
Fail cleanly	A clear, logged error with the request ID, so someone can investigate.
Never	Pass bad data downstream, or switch validation off.

ON THE EXAM

A validation failure is an expected path, not a terminal error. Design it like any other feature.

Skepticism toward confident output

- **Valid is not true.** A schema-perfect record can carry an invented figure.
- **Check values against the source:** ask for the supporting quote, or use citations in a separate call.
- **Cross-check fields:** end date after start date; totals that add up.
- **Self-reported confidence is a weak signal.** Do not let it replace a check.
- **Sample high-stakes outputs** for human review, and feed findings back into tests.

> ON THE EXAM
>
> The guide names this skill directly: be skeptical of confident output, and verify what decisions depend on.

Copperlet's document reader, fixed

BEFORE

Regex and hope

- JSON requested in the prompt
- Fields pulled out with regular expressions
- About 2% malformed or truncated
- Import job crashed on bad rows

AFTER

Shape, meaning, path

- Structured outputs from a Pydantic model
- Stop reason checked first
- Deposit, rent and date rules in code
- One retry with the error, then a person

SETTLED

The import job never sees invalid data now. Every failure has an owner.

Copperlet's triage labels, revisited

- **Enums fixed the vocabulary:** only seven trades and three urgencies can come back.
- **Enums do not fix the judgement:** a leak can still be labelled heating one day and plumbing the next.
- **That is a different layer:** clear category definitions, examples, and evidence from traces.
- **Module 30** diagnoses it properly.

> THE TIEBREAKER
>
> Structured outputs constrain which answers are possible, not which one is right.

> An app asks Claude for JSON in the prompt and passes the parsed result to a billing service, which rejects malformed input. About one response in a hundred fails to parse.
>
> The team wants the billing service never to receive malformed or invalid data.

What should the team do?

A. Add "RESPOND WITH VALID JSON ONLY" in capitals at the end of the prompt

B. Use structured outputs with the schema, and still validate before passing anything to billing

C. Let the billing service handle bad input, since it already rejects malformed data

D. Replace the parser with a more lenient one that extracts whatever looks like JSON

◆ PAUSE AND DECIDE

What should the team do?

A Add "RESPOND WITH VALID JSON ONLY" in capitals at the end of the prompt

Louder, still not guaranteed. The prompt is not an enforcement layer.

B Use structured outputs with the schema, and still validate before passing anything to billing

Guaranteed shape plus a meaning check. Covers both malformed and invalid.

C Let the billing service handle bad input, since it already rejects malformed data

Pushes your failure into someone else's system.

D Replace the parser with a more lenient one that extracts whatever looks like JSON

Lets more bad data through, not less.

THE SIGNAL

"Malformed or invalid": structured outputs answer the first word, validation the second.

> A lettings extraction uses structured outputs. Every response is valid JSON. But for agreements that never mention a deposit, the deposit_amount field still contains a plausible figure.
>
> The field is marked required, with type number.

What is the best fix?

A Lower the temperature so that Claude invents fewer numbers

B Keep the field required, so Claude is forced to look harder for a deposit

C Move to a larger model that is less likely to guess

D Let the field be null when the document is silent, say so in the prompt, and check values against the source

◆ PAUSE AND DECIDE

What is the best fix?

A Lower the temperature so that Claude invents fewer numbers

Wrong layer, and rejected on current models.

B Keep the field required, so Claude is forced to look harder for a deposit

Required with no 'unknown' is what forces the invention.

C Move to a larger model that is less likely to guess

The schema still demands a number. Any model must supply one.

D Let the field be null when the document is silent, say so in the prompt, and check values against the source

Gives Claude an honest answer to give, and verifies the rest.

THE SIGNAL

Valid JSON with an invented value: the schema left no honest way to say "not stated".

> A developer turns on structured outputs and proposes deleting all validation code, because "the output is now guaranteed".
>
> A reviewer disagrees.

In which two cases can a response fail to match the schema even with structured outputs on? Choose two.

A When the schema uses enums for its categories

B When the system prompt is longer than a few paragraphs

C When the response stops because it reached max tokens

D When Claude refuses the request

E When the same schema was already used the day before

◆ PAUSE AND DECIDE

In which two cases can a response fail to match the schema even with structured outputs on? Choose two.

A. When the schema uses enums for its categories
Enums are supported. Only letter case may differ.

B. When the system prompt is longer than a few paragraphs
Prompt length does not affect constrained decoding.

C. When the response stops because it reached max tokens
The JSON may be cut off part-way.

D. When Claude refuses the request
A refusal takes precedence over the schema.

E. When the same schema was already used the day before
Reuse is fine; the compiled grammar is simply cached.

THE SIGNAL

Check the stop reason first. Then validate. Keep the validation code.

> An app validates Claude's structured output and treats every validation failure as a fatal error, so the user's whole request fails.
>
> Failures are rare but painful, and the team wants them handled gracefully without weakening the checks.

What should the team do?

A. Treat failure as an expected path: retry once with the error message, then fall back to a person or a safe default

B. Switch validation off until the cause of the failures has been found

C. Pass outputs that fail validation downstream, and let each consumer decide what to do

D. Tell users that failures are unavoidable and ask them to check every output themselves

◆ PAUSE AND DECIDE

What should the team do?

A Treat failure as an expected path: retry once with the error message, then fall back to a person or a safe default

Graceful, and the checks stay exactly as strict.

B Switch validation off until the cause of the failures has been found

Weakens the checks, the one thing ruled out.

C Pass outputs that fail validation downstream, and let each consumer decide what to do

Spreads one failure across every consumer.

D Tell users that failures are unavoidable and ask them to check every output themselves

Pushes the problem onto users.

THE SIGNAL

"Without weakening the checks" rules out B and C. A designed failure path is the answer.

> A downstream reporting service crashes whenever a model output is missing an optional field, or contains a field it does not recognise.
>
> The owners want reliability without losing visibility of odd outputs.

What is the best approach in the code that reads the output?

A Silently drop any output that does not exactly match expectations

B Let the service crash and restart it automatically each time

C Parse defensively: handle missing and unknown fields on purpose, log them, and fail into a clear error path

D Trust the output completely, since structured outputs are switched on

◆ PAUSE AND DECIDE

What is the best approach in the code that reads the output?

A Silently drop any output that does not exactly match expectations

Reliable, but blind. Visibility was a stated need.

B Let the service crash and restart it automatically each time

Automates the outage instead of preventing it.

C Parse defensively: handle missing and unknown fields on purpose, log them, and fail into a clear error path

Reliable and visible: every odd case has a planned, logged outcome.

D Trust the output completely, since structured outputs are switched on

Optional fields can still be absent, and schemas change.

THE SIGNAL

Two needs, reliability and visibility. Only defensive parsing with logging meets both.

> Copperlet's triage uses structured outputs with an urgency enum of emergency, urgent and routine. Very occasionally a response contains "Urgent" with a capital letter, and the strict comparison in the queueing code fails.
>
> Everything else about these responses is correct.

What is the best fix?

A Lower the temperature so the model sticks to the exact lower-case values

B Normalise letter case in code before validating, and log how often it happens

C Remove the enum so that any string passes validation

D Switch structured outputs off and ask for the labels in the prompt

◆ PAUSE AND DECIDE

What is the best fix?

A Lower the temperature so the model sticks to the exact lower-case values

Not available on current models, and not the documented cause.

B Normalise letter case in code before validating, and log how often it happens

A documented quirk, repaired cheaply in code. The log keeps it visible.

C Remove the enum so that any string passes validation

Throws away the vocabulary control to fix capital letters.

D Switch structured outputs off and ask for the labels in the prompt

Trades a tiny quirk for every parse error back.

THE SIGNAL

A trivial, documented deviation is a repair job for code, not a redesign.

Module 7 in five lines

- **Code reads it? Use structured outputs:** a schema in the request, shape guaranteed.
- **Design the schema:** enums for categories, null for "not stated", no personal data in it.
- **Still validate:** stop reason, parse, schema, business rules - refusals and truncation break the shape.
- **Failure is a path:** retry with the error, repair trivia, fall back, or fail cleanly.
- **Valid is not true.** Check values against sources; be skeptical of confident output.

COMING UP

Next: context engineering - drift, bloat, pruning, compaction and isolation.